```
//@version=5
indicator("(5M) WAIT + ADX + RSI + CCI + REG SuperTrend [Regime • Exhaustion • Pullback Entry
• Sticky Labels] • CLEAN",
     overlay=true, max_labels_count=500, max_lines_count=500, max_boxes_count=200)

//
------------------------------------------------------------------------------------------------------------------------
// Inputs (original)
//
------------------------------------------------------------------------------------------------------------------------
emaFastLen   = input.int(5,  "EMA 5",  minval=1)
emaSlowLen   = input.int(20, "EMA 20", minval=1)
emaFilterLen = input.int(50, "EMA 50 (Bias)", minval=1)

rsiLen = input.int(14, "RSI Len", minval=2)
cciLen = input.int(20, "CCI Len", minval=5)

adxLen = input.int(14, "ADX Len", minval=2)
adxMin = input.float(20.0, "ADX Min (trend var)", step=0.5)

//
------------------------------------------------------------------------------------------------------------------------
// NEW: ADX MODE TEXT (Panelde “alınır/alınmaz” metni)
//
------------------------------------------------------------------------------------------------------------------------
grpAdxMode   = "ADX Mode (Panel Yazısı)"
showAdxMode  = input.bool(true, "Show ADX Mode text on panel", group=grpAdxMode)

adxT_no      = input.float(15.0, "T1: < (NO TRADE)", step=0.5, group=grpAdxMode)
adxT_weak    = input.float(20.0, "T2: < (WEAK)",     step=0.5, group=grpAdxMode)
adxT_ok      = input.float(25.0, "T3: < (OK)",       step=0.5, group=grpAdxMode)
adxT_clean   = input.float(35.0, "T4: < (CLEAN)",    step=0.5, group=grpAdxMode)

//
------------------------------------------------------------------------------------------------------------------------
// Language (TEXT ONLY)
//
------------------------------------------------------------------------------------------------------------------------
grpLang = "Language / Dil"
langOpt = input.string("EN", "Language", options=["EN","TR"], group=grpLang)
isTR    = langOpt == "TR"

//
------------------------------------------------------------------------------------------------------------------------
// UI Size (ONLY TABLE SIZE)
//
------------------------------------------------------------------------------------------------------------------------
grpUI      = "UI / Görünüm"
uiTextSize = input.string("Tiny", "Table Text Size", options=["Normal","Small","Tiny"],
group=grpUI)
```

```
tblSize    = uiTextSize == "Tiny" ? size.tiny : uiTextSize == "Small" ? size.small : size.normal

// Text dictionary
TXT_BUY     = isTR ? "AL" : "BUY"
TXT_SELL    = isTR ? "SAT" : "SELL"
TXT_UP      = isTR ? "YUKARI" : "UP"
TXT_DOWN    = isTR ? "AŞAĞI" : "DOWN"
TXT_NEUTRAL = isTR ? "NÖTR" : "NEUTRAL"
TXT_RISK_ON = isTR ? "RİSK AÇIK" : "RISK ON"
TXT_RISK_OFF= isTR ? "RİSK KAPALI" : "RISK OFF"

// Table headers
H_COIN     = isTR ? "COIN" : "COIN"
H_STATE    = isTR ? "DURUM" : "STATE"
H_REGSIG   = isTR ? "REG SİNYAL" : "REG SIGNAL"
H_REGTREND = isTR ? "REG TREND" : "REG TREND"
H_REGIME   = isTR ? "REJİM" : "REGIME"
H_EXH      = isTR ? "YORULMA" : "EXH"
H_USDTD    = "USDT.D"
H_CCI      = "CCI"
H_RSI      = "RSI"
H_ADX      = "ADX"

// Regime / Exhaustion labels
TXT_TREND    = isTR ? "TREND" : "TREND"
TXT_RANGE    = isTR ? "YATAY" : "RANGE"
TXT_EXH_OK   = isTR ? "OK" : "OK"
TXT_EXH_WARN = isTR ? "YORULUYOR" : "EXHAUST"

//
------------------------------------------------------------------------------
// Regression SuperTrend Inputs (original)
//
------------------------------------------------------------------------------
grpReg      = "Regression SuperTrend"
regLen      = input.int(20, "Regression Length", minval=5, group=grpReg)
rmseMul     = input.float(2, "RMSE Multiplier", step=0.1, group=grpReg)
regSrc      = input.source(close, "Source", group=grpReg)
showRegLine = input.bool(true, "Show Reg SuperTrend Line", group=grpReg)
upColor     = input.color(color.rgb(6, 162, 47), "Uptrend Color", group=grpReg)
downColor   = input.color(color.rgb(207, 23, 23), "Downtrend Color", group=grpReg)

//
------------------------------------------------------------------------------
// Regime Filter (Trend vs Range)
//
------------------------------------------------------------------------------
grpRegime        = "Regime Filter (TREND / RANGE)"
useRegimeFilter  = input.bool(true, "Use Regime Filter (suppress flips in RANGE)",
group=grpRegime)
regmFlipLookback = input.int(60, "Flip Lookback Bars", minval=20, maxval=300,
group=grpRegime)
regmFlipThresh   = input.int(4, "Flip Threshold (>= => RANGE)", minval=2, maxval=20,
group=grpRegime)
regmAdxRangeMax  = input.float(18.0, "ADX Max for RANGE", step=0.5, group=grpRegime)
regmSlopeLen     = input.int(8, "EMA50 Slope Len", minval=2, maxval=50, group=grpRegime)
```

```
regmSlopeMinPct  = input.float(0.03, "EMA50 Slope Min % (below => flat)", step=0.01,
group=grpRegime)

//
------------------------------------------------------------------------------------------
------------------------------------------------------------------------
// Exhaustion Warning (Trend Yoruluyor)
//
------------------------------------------------------------------------------------------
------------------------------------------------------------------------
grpExh       = "Exhaustion Warning (Trend Yoruluyor)"
useExhaustion = input.bool(true, "Show Exhaustion (fatigue) status in table", group=grpExh)
exhDistEmaLen = input.int(10, "Distance EMA Len", minval=3, maxval=50, group=grpExh)
exhCheckBars  = input.int(3, "Contraction Bars", minval=2, maxval=10, group=grpExh)
exhRsiBars    = input.int(3, "RSI Weakening Bars", minval=2, maxval=10, group=grpExh)

//
------------------------------------------------------------------------------------------
------------------------------------------------------------------------
// Auto Fibonacci (15m / 30m / 1H) — 4H REMOVED (as is)
// + Daily Fibonacci preserved
//
------------------------------------------------------------------------------------------
------------------------------------------------------------------------
grpFib    = "Auto Fibonacci (15m / 30m / 1H)"
fibEnable = input.bool(true, "Enable Auto Fibonacci", group=grpFib)

// Daily Fibonacci
grpFibD           = "Auto Fibonacci (Daily • Prev Day)"
fibDailyEnable    = input.bool(false, "Enable Daily Fibo (Prev Day High/Low)", group=grpFibD)
fibDailyOnlyIntra  = input.bool(true, "Show Daily Fibo only on Intraday charts", group=grpFibD)
fibDailyExtendBars = input.int(180, "Daily Fibo Right Extension (bars)", minval=20, maxval=2000,
group=grpFibD)
fibDailyShowBand   = input.bool(true, "Fill 0.5–0.618 Band (Daily)", group=grpFibD)
fibDailyLineWidth  = input.int(1, "Daily Line Width", minval=1, maxval=4, group=grpFibD)
fibDailyLineColor  = input.color(color.new(color.yellow, 0), "Daily Line Color", group=grpFibD)
fibDailyBandColor  = input.color(color.new(color.yellow, 85), "Daily Band Color", group=grpFibD)
fibDailyUtcOffsetH = input.int(-5, "Daily Day Boundary UTC Offset (hours)", minval=-12,
maxval=14, group=grpFibD)

// show only on these chart TFs
fibShow15m = input.bool(true, "Show on 15m", group=grpFib)
fibShow30m = input.bool(true, "Show on 30m", group=grpFib)
fibShow1H  = input.bool(true, "Show on 1H", group=grpFib)

fibLeft       = input.int(5, "Pivot Left Bars", minval=1, group=grpFib)
fibRight      = input.int(5, "Pivot Right Bars", minval=1, group=grpFib)
fibExtendBars = input.int(50, "Fib Right Extension (bars)", minval=5, maxval=300, group=grpFib)

fibLineColor   = input.color(color.new(color.white, 0), "Fib Line Color", group=grpFib)
fibLineWidth   = input.int(1, "Fib Line Width", minval=1, maxval=5, group=grpFib)
fibLineStyleIn = input.string("Solid", "Fib Line Style", options=["Solid","Dashed","Dotted"],
group=grpFib)

fibBandEnable = input.bool(true, "Fill 0.5–0.618 Band", group=grpFib)
fibBandColor  = input.color(color.new(color.teal, 75), "Band Color (0.5–0.618)", group=grpFib)

// Labels
fibShowLabels    = input.bool(true, "Show Fib Labels", group=grpFib)
fibLabelSet      = input.string("Key", "Label Set", options=["Key","All"], group=grpFib)
```

```
fibLabelFormat   = input.string("Full", "Label Format", options=["Full","Compact"], group=grpFib)
fibLabelTextCol  = input.color(color.new(color.white, 0), "Label Text Color", group=grpFib)
fibLabelBgCol    = input.color(color.new(color.black, 70), "Label Background", group=grpFib)
fibLabelSizeIn   = input.string("Small", "Label Size", options=["Tiny","Small","Normal"],
group=grpFib)
fibLabelOffset   = input.int(1, "Label X Offset (bars)", minval=0, maxval=5, group=grpFib)

fibLblSize = fibLabelSizeIn == "Tiny" ? size.tiny : fibLabelSizeIn == "Normal" ? size.normal :
size.small

//
------------------------------------------------------------------------------------------------
------------------------------------------------------------------------
// USDT.D (TABLE ONLY)
//
------------------------------------------------------------------------------------------------
------------------------------------------------------------------------
grpUSDT     = "USDT.D (Table Only)"
usdtRegLen  = input.int(50, "USDT.D Regression Len", minval=5, group=grpUSDT)
usdt_d      = request.security("CRYPTOCAP:USDT.D", timeframe.period, close)
usdt_reg    = ta.linreg(usdt_d, usdtRegLen, 0)
usdt_slope  = usdt_reg - usdt_reg[1]
usdtRiskOn  = usdt_slope < 0
usdtRiskOff = usdt_slope > 0
usdt_val_txt= str.tostring(usdt_d, "#.00")
usdt_text   = usdt_slope > 0 ? (TXT_RISK_OFF + " ↑ (" + usdt_val_txt + ")") : usdt_slope < 0 ?
(TXT_RISK_ON + " ↓ (" + usdt_val_txt + ")") : (TXT_NEUTRAL + " (" + usdt_val_txt + ")")
usdt_bg     = usdt_slope > 0 ? color.new(#f30505, 70) : usdt_slope < 0 ? color.new(#08f50f, 70) :
color.new(color.gray, 85)

//
------------------------------------------------------------------------------------------------
------------------------------------------------------------------------
// MANUAL ADX (Wilder)
//
------------------------------------------------------------------------------------------------
------------------------------------------------------------------------
f_tr() =>
    math.max(high - low, math.max(math.abs(high - close[1]), math.abs(low - close[1])))

f_adx(_len) =>
    upMove   = high - high[1]
    downMove = low[1] - low
    plusDM   = (upMove > downMove and upMove > 0) ? upMove : 0.0
    minusDM  = (downMove > upMove and downMove > 0) ? downMove : 0.0
    tr       = f_tr()
    smTR     = ta.rma(tr, _len)
    smPlusDM = ta.rma(plusDM, _len)
    smMinusDM= ta.rma(minusDM, _len)
    plusDI   = smTR == 0 ? 0 : 100 * smPlusDM / smTR
    minusDI  = smTR == 0 ? 0 : 100 * smMinusDM / smTR
    dx       = (plusDI + minusDI) == 0 ? 0 : 100 * math.abs(plusDI - minusDI) / (plusDI + minusDI)
    ta.rma(dx, _len)
//
------------------------------------------------------------------------------------------------
------------------------------------------------------------------------
// Break Quality Filter (Fake breakout filtresi)
// SOFT (yumuşak): Engellemez, sadece etiket metninde "WEAK" ekler.
// HARD (sert): Zayıf kırılımda ENTRY/CHOCH kurulumunu engeller.
// body ratio (gövde oranı) = |C-O| / (H-L)
```

```
// ATR (Average True Range / ortalama gerçek aralık) ile gövde büyüklüğü kontrolü
//
------------------------------------------------------------------------------------------------------------------------

grpBQ = "Break Quality Filter (Fake Breakout)"
bqMode = input.string("SOFT", "Mode", options=["OFF","SOFT","HARD"], group=grpBQ)

bqBodyRatioMinSoft = input.float(0.35, "SOFT: Min Body Ratio", step=0.01, minval=0.05,
maxval=0.95, group=grpBQ)
bqBodyAtrMinSoft   = input.float(0.10, "SOFT: Min Body (ATR x)", step=0.01, minval=0.00,
maxval=2.00, group=grpBQ)

bqBodyRatioMinHard = input.float(0.50, "HARD: Min Body Ratio", step=0.01, minval=0.05,
maxval=0.95, group=grpBQ)
bqBodyAtrMinHard   = input.float(0.18, "HARD: Min Body (ATR x)", step=0.01, minval=0.00,
maxval=2.00, group=grpBQ)

// Global ATR (tek yerden)
bqATR = ta.atr(14)

// Kırılım mumu kalite metriği
bqRange = math.max(high - low, syminfo.mintick)
bqBody  = math.abs(close - open)
bqBodyRatio = bqBody / bqRange

// Eşikler
bqMinRatio = bqMode == "HARD" ? bqBodyRatioMinHard : bqBodyRatioMinSoft
bqMinAtr   = bqMode == "HARD" ? bqBodyAtrMinHard   : bqBodyAtrMinSoft

bqOk = (bqMode == "OFF") ? true : (bqBodyRatio >= bqMinRatio and bqBody >= (bqATR *
bqMinAtr))

bqWhy =
    bqMode == "OFF" ? "BQ:OFF" :
    (bqBodyRatio < bqMinRatio ? "BQ:LOW_BODY_RATIO" : (bqBody < bqATR * bqMinAtr ?
"BQ:LOW_BODY_ATR" : "BQ:OK"))

// Flip bazlı kullanmak için “son flip” kalitesini sakla
var bool   lastFlipBqOk  = true
var string lastFlipBqWhy = "BQ:OK"
//
------------------------------------------------------------------------------------------------------------------------

// Core
//
------------------------------------------------------------------------------------------------------------------------

ema50  = ta.ema(close, emaFilterLen)
rsiVal = ta.rsi(close, rsiLen)
cciVal = ta.cci(hlc3, cciLen)
adxVal = f_adx(adxLen)

bull  = close > ema50
bear  = close < ema50
state = bull and adxVal >= adxMin ? "WAIT(L)" : bear and adxVal >= adxMin ? "WAIT(S)" : "WAIT"

//
------------------------------------------------------------------------------------------------------------------------

// Regression SuperTrend
```

```
//
// ---------------------------------------------------------------------------------------------------------------------
// ---------------------------------------------------------------------------------------------------------------------
regLine = ta.linreg(regSrc, regLen, 0)

// RMSE
float err = 0.0
for i = 0 to regLen - 1
    err += math.pow(regSrc[i] - regLine[i], 2)

rmse      = math.sqrt(err / (regLen - 1))
upper_raw = regLine + rmse * rmseMul
lower_raw = regLine - rmse * rmseMul

var float upper = na
var float lower = na
var int regTrend = 1

upper := na(upper[1]) ? upper_raw : (regTrend == -1 ? math.min(upper_raw, upper[1]) : upper_raw)
lower := na(lower[1]) ? lower_raw : (regTrend == 1 ? math.max(lower_raw, lower[1]) : lower_raw)

prevTrend = regTrend
regTrend := regTrend == 1 and close < lower ? -1 : regTrend == -1 and close > upper ? 1 :
regTrend
flip      = regTrend != prevTrend
regST     = regTrend == 1 ? lower : upper

//
// ---------------------------------------------------------------------------------------------------------------------
// ---------------------------------------------------------------------------------------------------------------------
// Regime classification (TREND vs RANGE)
//
// ---------------------------------------------------------------------------------------------------------------------
// ---------------------------------------------------------------------------------------------------------------------
emaSlope    = ema50 - ema50[regmSlopeLen]
emaSlopePct = ema50 != 0 ? math.abs(emaSlope) / ema50 * 100.0 : 0.0

int flipCount = 0
for i = 0 to regmFlipLookback - 1
    flipCount += flip[i] ? 1 : 0

flatSlope      = emaSlopePct < regmSlopeMinPct
lowAdxForRange = adxVal <= regmAdxRangeMax
isRange        = (flipCount >= regmFlipThresh and lowAdxForRange) or (flatSlope and
lowAdxForRange)
regimeTxt      = isRange ? TXT_RANGE : TXT_TREND

allowFlip = not useRegimeFilter or not isRange
//
// ---------------------------------------------------------------------------------------------------------------------
// ---------------------------------------------------------------------------------------------------------------------
// AUTO MODE (TREND vs RANGE) + MODE ROW TEXT
// Bu blok: ENTRY parametrelerini rejime göre otomatik seçer + panelde gösterir
//
// ---------------------------------------------------------------------------------------------------------------------
// ---------------------------------------------------------------------------------------------------------------------
grpAutoMode = "AUTO MODE (TREND/RANGE)"
entryTolAtrMult  = input.float(0.15, "FALLBACK: Touch Tol (ATR x)",       step=0.01, minval=0.0,
maxval=1.0, group=grpAutoMode)
```

```
entryRetrAtrMult = input.float(0.25, "FALLBACK: Require Retrace (ATR x)", step=0.01, minval=0.0,
maxval=3.0, group=grpAutoMode)
// Modu panelde göster
showModeRow = input.bool(true, "Show MODE row on panel", group=grpAutoMode)

// ENTRY AUTO MODE aktif mi?
useAutoModeForEntry = input.bool(true, "Auto-switch ENTRY params by regime",
group=grpAutoMode)

// ——————————————————————————
// FALLBACK (AUTO KAPALIYSA KULLANILACAK) - ESKİ ENTRY INPUTLARI
// Not: Bu iki input daha aşağıda zaten varsa, ORADAKİLERİ SİL, sadece burada kalsın.
// ——————————————————————————

// ——————————————————————————
// TREND param seti (trend devamı için)
// ——————————————————————————
trend_entryTolAtrMult  = input.float(0.15, "TREND: Touch Tol (ATR x)",       step=0.01, minval=0.0,
maxval=1.0, group=grpAutoMode)
trend_entryRetrAtrMult = input.float(0.25, "TREND: Require Retrace (ATR x)", step=0.01,
minval=0.0, maxval=3.0, group=grpAutoMode)

// ——————————————————————————
// RANGE param seti (yatay için) — daha seçici, daha az sinyal, daha az fake (sahte)
// ——————————————————————————
range_entryTolAtrMult  = input.float(0.08, "RANGE: Touch Tol (ATR x)",       step=0.01, minval=0.0,
maxval=1.0, group=grpAutoMode)
range_entryRetrAtrMult = input.float(0.35, "RANGE: Require Retrace (ATR x)", step=0.01,
minval=0.0, maxval=3.0, group=grpAutoMode)

// Aktif mod seçimi
activeIsRange = isRange
activeModeTxt = activeIsRange ? (isTR ? "MOD: YATAY" : "MODE: RANGE") : (isTR ? "MOD:
TREND" : "MODE: TREND")

// Etkin (effective) değerler:
// AUTO kapalıysa → entryTolAtrMult / entryRetrAtrMult
// AUTO açıksa   → TREND veya RANGE seti
entryTol_eff  = useAutoModeForEntry ? (activeIsRange ? range_entryTolAtrMult  :
trend_entryTolAtrMult)  : entryTolAtrMult
entryRetr_eff = useAutoModeForEntry ? (activeIsRange ? range_entryRetrAtrMult :
trend_entryRetrAtrMult) : entryRetrAtrMult
//
——————————————————————————————————————————————————————————————————————
——————————————————————————————————————————————————————————————————————
// NEW: Flip candle BODY line (thin) — keeps last N lines
//
——————————————————————————————————————————————————————————————————————
——————————————————————————————————————————————————————————————————————
grpFlipLine   = "Flip Body Line (ince gövde çizgisi)"
flipLineOn    = input.bool(true, "Enable Flip Body Line", group=grpFlipLine)
flipLineKeep  = input.int(120, "Keep last N flip lines", minval=10, maxval=400, group=grpFlipLine)
flipLineWidth = input.int(1, "Line width", minval=1, maxval=3, group=grpFlipLine)

var array<line> flipLines = array.new_line()

if flipLineOn and flip and allowFlip and barstate.isconfirmed
    y1 = open
    y2 = close
    col = regTrend == 1 ? color.new(upColor, 0) : color.new(downColor, 0)
```

```
    l = line.new(bar_index, y1, bar_index, y2, xloc=xloc.bar_index, extend=extend.none, color=col,
width=flipLineWidth)
    array.push(flipLines, l)
    if array.size(flipLines) > flipLineKeep
        old = array.shift(flipLines)
        if not na(old)
            line.delete(old)

//
------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------
// AUTO FIB ENGINE (15m / 30m / 1H) — AS IS
//
------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------
fib_is15 = timeframe.isminutes and timeframe.multiplier == 15
fib_is30 = timeframe.isminutes and timeframe.multiplier == 30
fib_is60 = timeframe.isminutes and timeframe.multiplier == 60
//
------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------
// AUTO PIVOT (TF'ye göre otomatik)
// 15m => 5/5
// 30m => 10/10
// diğer TF'ler => input'taki fibLeft/fibRight
//
------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------
int fibLeftEff  = fib_is15 ? 5 : fib_is30 ? 10 : fibLeft
int fibRightEff = fib_is15 ? 5 : fib_is30 ? 10 : fibRight
fib_okTF = (fibShow15m and fib_is15) or (fibShow30m and fib_is30) or (fibShow1H and fib_is60)

fibLineStyle = fibLineStyleIn == "Dashed" ? line.style_dashed : fibLineStyleIn == "Dotted" ?
line.style_dotted : line.style_solid

fib_pH     = ta.pivothigh(high, fibLeftEff, fibRightEff)
fib_pL     = ta.pivotlow(low,  fibLeftEff, fibRightEff)
fib_pH_bar = bar_index - fibRightEff
fib_pL_bar = bar_index - fibRightEff

var float fib_lastHigh   = na
var int   fib_lastHighBar= na
var float fib_lastLow    = na
var int   fib_lastLowBar = na

if not na(fib_pH)
    fib_lastHigh    := fib_pH
    fib_lastHighBar := fib_pH_bar
if not na(fib_pL)
    fib_lastLow    := fib_pL
    fib_lastLowBar := fib_pL_bar

lvl_m2    = -2.0
lvl_m1618 = -1.618
lvl_m1272 = -1.272
lvl_m1    = -1.0
lvl_m618  = -0.618
lvl_0     = 0.0
lvl_236   = 0.236
lvl_382   = 0.382
```

```
lvl_5     = 0.5
lvl_618   = 0.618
lvl_786   = 0.786
lvl_1     = 1.0
lvl_1272  = 1.272
lvl_1414  = 1.414
lvl_1618  = 1.618
lvl_2     = 2.0

var line     fibL_m2     = na
var line     fibL_m1618  = na
var line     fibL_m1272  = na
var line     fibL_m1     = na
var line     fibL_m618   = na
var line     fibL_0      = na
var line     fibL_236    = na
var line     fibL_382    = na
var line     fibL_5      = na
var line     fibL_618    = na
var line     fibL_786    = na
var line     fibL_1      = na
var line     fibL_1272   = na
var line     fibL_1414   = na
var line     fibL_1618   = na
var line     fibL_2      = na
var linefill fibFill_5_618 = na

var label fibLb_m2    = na
var label fibLb_m1618 = na
var label fibLb_m1272 = na
var label fibLb_m1    = na
var label fibLb_m618  = na
var label fibLb_0     = na
var label fibLb_236   = na
var label fibLb_382   = na
var label fibLb_5     = na
var label fibLb_618   = na
var label fibLb_786   = na
var label fibLb_1     = na
var label fibLb_1272  = na
var label fibLb_1414  = na
var label fibLb_1618  = na
var label fibLb_2     = na

var int   fib_drawFromBar = na
var float fib_drawFromPx  = na
var int   fib_drawToBar   = na
var float fib_drawToPx    = na

fib_lbl_text(_lvlTxt, _price) =>
    fibLabelFormat == "Compact" ? _lvlTxt : (_lvlTxt + " " + str.tostring(_price, format.mintick))

drawAll = fibLabelSet == "All"

if fibEnable and (flip or not fib_okTF)
    if not na(fibL_m2)
        line.delete(fibL_m2)
    if not na(fibL_m1618)
        line.delete(fibL_m1618)
    if not na(fibL_m1272)
```

```
    line.delete(fibL_m1272)
if not na(fibL_m1)
    line.delete(fibL_m1)
if not na(fibL_m618)
    line.delete(fibL_m618)
if not na(fibL_0)
    line.delete(fibL_0)
if not na(fibL_236)
    line.delete(fibL_236)
if not na(fibL_382)
    line.delete(fibL_382)
if not na(fibL_5)
    line.delete(fibL_5)
if not na(fibL_618)
    line.delete(fibL_618)
if not na(fibL_786)
    line.delete(fibL_786)
if not na(fibL_1)
    line.delete(fibL_1)
if not na(fibL_1272)
    line.delete(fibL_1272)
if not na(fibL_1414)
    line.delete(fibL_1414)
if not na(fibL_1618)
    line.delete(fibL_1618)
if not na(fibL_2)
    line.delete(fibL_2)
if not na(fibFill_5_618)
    linefill.delete(fibFill_5_618)

if not na(fibLb_m2)
    label.delete(fibLb_m2)
if not na(fibLb_m1618)
    label.delete(fibLb_m1618)
if not na(fibLb_m1272)
    label.delete(fibLb_m1272)
if not na(fibLb_m1)
    label.delete(fibLb_m1)
if not na(fibLb_m618)
    label.delete(fibLb_m618)
if not na(fibLb_0)
    label.delete(fibLb_0)
if not na(fibLb_236)
    label.delete(fibLb_236)
if not na(fibLb_382)
    label.delete(fibLb_382)
if not na(fibLb_5)
    label.delete(fibLb_5)
if not na(fibLb_618)
    label.delete(fibLb_618)
if not na(fibLb_786)
    label.delete(fibLb_786)
if not na(fibLb_1)
    label.delete(fibLb_1)
if not na(fibLb_1272)
    label.delete(fibLb_1272)
if not na(fibLb_1414)
    label.delete(fibLb_1414)
if not na(fibLb_1618)
    label.delete(fibLb_1618)
```

```
    if not na(fibLb_2)
        label.delete(fibLb_2)

    fibL_m2 := na
    fibL_m1618 := na
    fibL_m1272 := na
    fibL_m1 := na
    fibL_m618 := na
    fibL_0 := na
    fibL_236 := na
    fibL_382 := na
    fibL_5 := na
    fibL_618 := na
    fibL_786 := na
    fibL_1 := na
    fibL_1272 := na
    fibL_1414 := na
    fibL_1618 := na
    fibL_2 := na
    fibFill_5_618 := na

    fibLb_m2 := na
    fibLb_m1618 := na
    fibLb_m1272 := na
    fibLb_m1 := na
    fibLb_m618 := na
    fibLb_0 := na
    fibLb_236 := na
    fibLb_382 := na
    fibLb_5 := na
    fibLb_618 := na
    fibLb_786 := na
    fibLb_1 := na
    fibLb_1272 := na
    fibLb_1414 := na
    fibLb_1618 := na
    fibLb_2 := na

    fib_drawFromBar := na
    fib_drawFromPx := na
    fib_drawToBar := na
    fib_drawToPx := na

fib_can = fibEnable and fib_okTF and not na(fib_lastHigh) and not na(fib_lastLow) and not
na(fib_lastHighBar) and not na(fib_lastLowBar)

int   wantFromBar = na
float wantFromPx  = na
int   wantToBar   = na
float wantToPx    = na

if fib_can
    upLegOk = fib_lastLowBar < fib_lastHighBar
    dnLegOk = fib_lastHighBar < fib_lastLowBar

    if regTrend == 1 and upLegOk
        wantFromBar := fib_lastLowBar
        wantFromPx  := fib_lastLow
        wantToBar   := fib_lastHighBar
        wantToPx    := fib_lastHigh
```

```
    if regTrend == -1 and dnLegOk
        wantFromBar := fib_lastHighBar
        wantFromPx  := fib_lastHigh
        wantToBar   := fib_lastLowBar
        wantToPx    := fib_lastLow

haveAnchors   = not na(wantFromBar) and not na(wantToBar) and not na(wantFromPx) and not
na(wantToPx)
firstDraw     = na(fibL_0)
anchorsChanged= haveAnchors and ((wantFromBar != fib_drawFromBar) or (wantToBar !=
fib_drawToBar) or (wantFromPx != fib_drawFromPx) or (wantToPx != fib_drawToPx))

if haveAnchors and (firstDraw or anchorsChanged)
    // delete existing
    if not na(fibL_m2)
        line.delete(fibL_m2)
    if not na(fibL_m1618)
        line.delete(fibL_m1618)
    if not na(fibL_m1272)
        line.delete(fibL_m1272)
    if not na(fibL_m1)
        line.delete(fibL_m1)
    if not na(fibL_m618)
        line.delete(fibL_m618)
    if not na(fibL_0)
        line.delete(fibL_0)
    if not na(fibL_236)
        line.delete(fibL_236)
    if not na(fibL_382)
        line.delete(fibL_382)
    if not na(fibL_5)
        line.delete(fibL_5)
    if not na(fibL_618)
        line.delete(fibL_618)
    if not na(fibL_786)
        line.delete(fibL_786)
    if not na(fibL_1)
        line.delete(fibL_1)
    if not na(fibL_1272)
        line.delete(fibL_1272)
    if not na(fibL_1414)
        line.delete(fibL_1414)
    if not na(fibL_1618)
        line.delete(fibL_1618)
    if not na(fibL_2)
        line.delete(fibL_2)
    if not na(fibFill_5_618)
        linefill.delete(fibFill_5_618)

    if not na(fibLb_m2)
        label.delete(fibLb_m2)
    if not na(fibLb_m1618)
        label.delete(fibLb_m1618)
    if not na(fibLb_m1272)
        label.delete(fibLb_m1272)
    if not na(fibLb_m1)
        label.delete(fibLb_m1)
    if not na(fibLb_m618)
        label.delete(fibLb_m618)
```

```
if not na(fibLb_0)
    label.delete(fibLb_0)
if not na(fibLb_236)
    label.delete(fibLb_236)
if not na(fibLb_382)
    label.delete(fibLb_382)
if not na(fibLb_5)
    label.delete(fibLb_5)
if not na(fibLb_618)
    label.delete(fibLb_618)
if not na(fibLb_786)
    label.delete(fibLb_786)
if not na(fibLb_1)
    label.delete(fibLb_1)
if not na(fibLb_1272)
    label.delete(fibLb_1272)
if not na(fibLb_1414)
    label.delete(fibLb_1414)
if not na(fibLb_1618)
    label.delete(fibLb_1618)
if not na(fibLb_2)
    label.delete(fibLb_2)

// reset handles
fibL_m2 := na
fibL_m1618 := na
fibL_m1272 := na
fibL_m1 := na
fibL_m618 := na
fibL_0 := na
fibL_236 := na
fibL_382 := na
fibL_5 := na
fibL_618 := na
fibL_786 := na
fibL_1 := na
fibL_1272 := na
fibL_1414 := na
fibL_1618 := na
fibL_2 := na
fibFill_5_618 := na

fibLb_m2 := na
fibLb_m1618 := na
fibLb_m1272 := na
fibLb_m1 := na
fibLb_m618 := na
fibLb_0 := na
fibLb_236 := na
fibLb_382 := na
fibLb_5 := na
fibLb_618 := na
fibLb_786 := na
fibLb_1 := na
fibLb_1272 := na
fibLb_1414 := na
fibLb_1618 := na
fibLb_2 := na

// window
```

```
    x2 = bar_index + fibExtendBars
    x1 = math.max(wantFromBar, bar_index - fibExtendBars)
    lx = x2 + fibLabelOffset

    rng   = (wantToPx - wantFromPx)
    pm2   = wantFromPx + rng * lvl_m2
    pm1618= wantFromPx + rng * lvl_m1618
    pm1272= wantFromPx + rng * lvl_m1272
    pm1   = wantFromPx + rng * lvl_m1
    pm618 = wantFromPx + rng * lvl_m618
    p0    = wantFromPx + rng * lvl_0
    p236  = wantFromPx + rng * lvl_236
    p382  = wantFromPx + rng * lvl_382
    p5    = wantFromPx + rng * lvl_5
    p618  = wantFromPx + rng * lvl_618
    p786  = wantFromPx + rng * lvl_786
    p1    = wantFromPx + rng * lvl_1
    p1272 = wantFromPx + rng * lvl_1272
    p1414 = wantFromPx + rng * lvl_1414
    p1618 = wantFromPx + rng * lvl_1618
    p2    = wantFromPx + rng * lvl_2

    if drawAll
        fibL_m2    := line.new(x1, pm2,    x2, pm2,    extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
        fibL_m1618 := line.new(x1, pm1618, x2, pm1618, extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
        fibL_m1272 := line.new(x1, pm1272, x2, pm1272, extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
        fibL_m1    := line.new(x1, pm1,    x2, pm1,    extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
        fibL_m618  := line.new(x1, pm618,  x2, pm618,  extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
        fibL_236   := line.new(x1, p236,   x2, p236,   extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
        fibL_1272  := line.new(x1, p1272,  x2, p1272,  extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
        fibL_1414  := line.new(x1, p1414,  x2, p1414,  extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
        fibL_1618  := line.new(x1, p1618,  x2, p1618,  extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
        fibL_2     := line.new(x1, p2,     x2, p2,     extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)

    fibL_0   := line.new(x1, p0,   x2, p0,   extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
    fibL_382 := line.new(x1, p382, x2, p382, extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
    fibL_5   := line.new(x1, p5,   x2, p5,   extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
    fibL_618 := line.new(x1, p618, x2, p618, extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
    fibL_786 := line.new(x1, p786, x2, p786, extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)
    fibL_1   := line.new(x1, p1,   x2, p1,   extend=extend.none, color=fibLineColor,
width=fibLineWidth, style=fibLineStyle)

    if fibBandEnable and not na(fibL_5) and not na(fibL_618)
        fibFill_5_618 := linefill.new(fibL_5, fibL_618, color=fibBandColor)
```

```
    if fibShowLabels
        if drawAll
            fibLb_m2    := label.new(lx, pm2,    fib_lbl_text("-2", pm2), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol,
size=fibLblSize)
            fibLb_m1618 := label.new(lx, pm1618, fib_lbl_text("-1.618", pm1618), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol,
size=fibLblSize)
            fibLb_m1272 := label.new(lx, pm1272, fib_lbl_text("-1.272", pm1272), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol,
size=fibLblSize)
            fibLb_m1    := label.new(lx, pm1,    fib_lbl_text("-1", pm1), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol,
size=fibLblSize)
            fibLb_m618  := label.new(lx, pm618,  fib_lbl_text("-0.618", pm618), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol,
size=fibLblSize)

        fibLb_0     := label.new(lx, p0,    fib_lbl_text("0", p0), xloc=xloc.bar_index, yloc=yloc.price,
style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol, size=fibLblSize)
        fibLb_236   := label.new(lx, p236,  fib_lbl_text("0.236", p236), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol,
size=fibLblSize)
        fibLb_1     := label.new(lx, p1,    fib_lbl_text("1", p1), xloc=xloc.bar_index, yloc=yloc.price,
style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol, size=fibLblSize)
        fibLb_1272  := label.new(lx, p1272, fib_lbl_text("1.272", p1272), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol,
size=fibLblSize)
        fibLb_1414  := label.new(lx, p1414, fib_lbl_text("1.414", p1414), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol,
size=fibLblSize)
        fibLb_1618  := label.new(lx, p1618, fib_lbl_text("1.618", p1618), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol,
size=fibLblSize)
        fibLb_2     := label.new(lx, p2,    fib_lbl_text("2", p2), xloc=xloc.bar_index, yloc=yloc.price,
style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol, size=fibLblSize)
        fibLb_382   := label.new(lx, p382,  fib_lbl_text("0.382", p382), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol,
size=fibLblSize)
        fibLb_5     := label.new(lx, p5,    fib_lbl_text("0.5", p5), xloc=xloc.bar_index, yloc=yloc.price,
style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol, size=fibLblSize)
        fibLb_618   := label.new(lx, p618,  fib_lbl_text("0.618", p618), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol,
size=fibLblSize)
        fibLb_786   := label.new(lx, p786,  fib_lbl_text("0.786", p786), xloc=xloc.bar_index,
yloc=yloc.price, style=label.style_label_left, textcolor=fibLabelTextCol, color=fibLabelBgCol,
size=fibLblSize)

    fib_drawFromBar := wantFromBar
    fib_drawFromPx  := wantFromPx
    fib_drawToBar   := wantToBar
    fib_drawToPx    := wantToPx

if fibEnable and fib_okTF and not na(fibL_0)
    newX2 = bar_index + fibExtendBars
    newX1 = na(fib_drawFromBar) ? (bar_index - fibExtendBars) : math.max(fib_drawFromBar,
bar_index - fibExtendBars)
    newLX = newX2 + fibLabelOffset

    if not na(fibL_m2)
```

```
    line.set_x1(fibL_m2, newX1)
    line.set_x2(fibL_m2, newX2)
if not na(fibL_m1618)
    line.set_x1(fibL_m1618, newX1)
    line.set_x2(fibL_m1618, newX2)
if not na(fibL_m1272)
    line.set_x1(fibL_m1272, newX1)
    line.set_x2(fibL_m1272, newX2)
if not na(fibL_m1)
    line.set_x1(fibL_m1, newX1)
    line.set_x2(fibL_m1, newX2)
if not na(fibL_m618)
    line.set_x1(fibL_m618, newX1)
    line.set_x2(fibL_m618, newX2)
if not na(fibL_0)
    line.set_x1(fibL_0, newX1)
    line.set_x2(fibL_0, newX2)
if not na(fibL_236)
    line.set_x1(fibL_236, newX1)
    line.set_x2(fibL_236, newX2)
if not na(fibL_382)
    line.set_x1(fibL_382, newX1)
    line.set_x2(fibL_382, newX2)
if not na(fibL_5)
    line.set_x1(fibL_5, newX1)
    line.set_x2(fibL_5, newX2)
if not na(fibL_618)
    line.set_x1(fibL_618, newX1)
    line.set_x2(fibL_618, newX2)
if not na(fibL_786)
    line.set_x1(fibL_786, newX1)
    line.set_x2(fibL_786, newX2)
if not na(fibL_1)
    line.set_x1(fibL_1, newX1)
    line.set_x2(fibL_1, newX2)
if not na(fibL_1272)
    line.set_x1(fibL_1272, newX1)
    line.set_x2(fibL_1272, newX2)
if not na(fibL_1414)
    line.set_x1(fibL_1414, newX1)
    line.set_x2(fibL_1414, newX2)
if not na(fibL_1618)
    line.set_x1(fibL_1618, newX1)
    line.set_x2(fibL_1618, newX2)
if not na(fibL_2)
    line.set_x1(fibL_2, newX1)
    line.set_x2(fibL_2, newX2)

if fibShowLabels
    if not na(fibLb_m2)
        label.set_x(fibLb_m2, newLX)
    if not na(fibLb_m1618)
        label.set_x(fibLb_m1618, newLX)
    if not na(fibLb_m1272)
        label.set_x(fibLb_m1272, newLX)
    if not na(fibLb_m1)
        label.set_x(fibLb_m1, newLX)
    if not na(fibLb_m618)
        label.set_x(fibLb_m618, newLX)
    if not na(fibLb_0)
```

```
            label.set_x(fibLb_0, newLX)
        if not na(fibLb_236)
            label.set_x(fibLb_236, newLX)
        if not na(fibLb_382)
            label.set_x(fibLb_382, newLX)
        if not na(fibLb_5)
            label.set_x(fibLb_5, newLX)
        if not na(fibLb_618)
            label.set_x(fibLb_618, newLX)
        if not na(fibLb_786)
            label.set_x(fibLb_786, newLX)
        if not na(fibLb_1)
            label.set_x(fibLb_1, newLX)
        if not na(fibLb_1272)
            label.set_x(fibLb_1272, newLX)
        if not na(fibLb_1414)
            label.set_x(fibLb_1414, newLX)
        if not na(fibLb_1618)
            label.set_x(fibLb_1618, newLX)
        if not na(fibLb_2)
            label.set_x(fibLb_2, newLX)

//
------------------------------------------------------------------------------------------------------------------------
// DAILY FIB ENGINE (Prev Day High/Low) — preserved
//
------------------------------------------------------------------------------------------------------------------------
canShowDaily = fibDailyEnable and (not fibDailyOnlyIntra or timeframe.isintraday)

dayMs = 86400000
offMs = fibDailyUtcOffsetH * 3600000

dayId = int(math.floor((time + offMs) / dayMs))
newDay = barstate.isfirst ? true : (dayId != dayId[1])

var float curDH = na
var float curDL = na
var float prevDH = na
var float prevDL = na

if newDay
    prevDH := curDH
    prevDL := curDL
    curDH := high
    curDL := low
else
    curDH := na(curDH) ? high : math.max(curDH, high)
    curDL := na(curDL) ? low  : math.min(curDL, low)

pdh = prevDH
pdl = prevDL

dT0 = (dayId * dayMs) - offMs

dRange = math.max(pdh - pdl, syminfo.mintick)
d0    = pdl
d50   = pdl + dRange * 0.50
d618  = pdl + dRange * 0.618
```

```
d1    = pdh

var line dL0   = na
var line dL50  = na
var line dL618 = na
var line dL1   = na
var box  dBand = na

secPerBar = timeframe.in_seconds(timeframe.period)
secSafe   = na(secPerBar) or secPerBar <= 0 ? 60 : secPerBar
extMs     = fibDailyExtendBars * secSafe * 1000

x1t = dT0
x2t = dT0 + extMs

if canShowDaily and not na(pdh) and not na(pdl) and not na(dT0) and (barstate.isfirst or newDay)
    if not na(dL0)
        line.delete(dL0)
    if not na(dL50)
        line.delete(dL50)
    if not na(dL618)
        line.delete(dL618)
    if not na(dL1)
        line.delete(dL1)
    if not na(dBand)
        box.delete(dBand)

    dL0   := line.new(x1t, d0,   x2t, d0,   xloc=xloc.bar_time, extend=extend.none,
color=fibDailyLineColor, width=fibDailyLineWidth)
    dL50  := line.new(x1t, d50,  x2t, d50,  xloc=xloc.bar_time, extend=extend.none,
color=fibDailyLineColor, width=fibDailyLineWidth)
    dL618 := line.new(x1t, d618, x2t, d618, xloc=xloc.bar_time, extend=extend.none,
color=fibDailyLineColor, width=fibDailyLineWidth)
    dL1   := line.new(x1t, d1,   x2t, d1,   xloc=xloc.bar_time, extend=extend.none,
color=fibDailyLineColor, width=fibDailyLineWidth)

    if fibDailyShowBand
        top = math.max(d50, d618)
        bot = math.min(d50, d618)
        dBand := box.new(left=x1t, top=top, right=x2t, bottom=bot, xloc=xloc.bar_time,
bgcolor=fibDailyBandColor, border_color=color.new(fibDailyLineColor, 65))

//
─────────────────────────────────────────────────────────────────────────────────────
// Exhaustion / fatigue detection
//
─────────────────────────────────────────────────────────────────────────────────────
distToST      = math.abs(close - regST)
distEma       = ta.ema(distToST, exhDistEmaLen)
distContract  = ta.falling(distEma, exhCheckBars)
rsiWeakUp     = ta.falling(rsiVal, exhRsiBars)
rsiWeakDown   = ta.rising(rsiVal, exhRsiBars)
exhaustUp     = regTrend == 1 and distContract and rsiWeakUp
exhaustDown   = regTrend == -1 and distContract and rsiWeakDown
exhausting    = useExhaustion and (exhaustUp or exhaustDown)
exhTxt        = not useExhaustion ? "—" : exhausting ? TXT_EXH_WARN : TXT_EXH_OK
```

```
//
// ---------------------------------------------------------------------------------------------------------------------
// REG SuperTrend – Glow effect (as is)
//
// ---------------------------------------------------------------------------------------------------------------------
trendCol   = regTrend == 1 ? upColor : downColor
baseCol    = color.new(trendCol, 0)
glowCol1   = color.new(trendCol, 75)
glowCol2   = color.new(trendCol, 88)
lineSeries = showRegLine and not flip ? regST : na

plot(lineSeries, color=glowCol2, linewidth=13, style=plot.style_linebr)
plot(lineSeries, color=glowCol1, linewidth=11, style=plot.style_linebr)
plot(lineSeries, color=baseCol,  linewidth=3,  style=plot.style_linebr)

//
// ---------------------------------------------------------------------------------------------------------------------
// CCI READY / PREP (ICT-style) — preserved
//
// ---------------------------------------------------------------------------------------------------------------------
grpPrep = "CCI READY / PREP (ICT-style)"
usePrepBox = input.bool(true, "Show CCI READY/PREP boxes", group=grpPrep)
prepExtreme = input.int(200, "Extreme level (±) to ARM", minval=100, maxval=400, step=10,
group=grpPrep)
prepTrigger = input.int(100, "Trigger level (±) to FIRE", minval=50, maxval=200, step=10,
group=grpPrep)
prepCooldown = input.int(12, "Cooldown bars", minval=0, maxval=200, group=grpPrep)
prepUseStateFilter = input.bool(true, "Filter PREP by STATE (WAIT(L)/WAIT(S))", group=grpPrep)

string TXT_PREP_L = isTR ? "HAZIR-L" : "PREP-L"
string TXT_PREP_S = isTR ? "HAZIR-S" : "PREP-S"

var bool prepLongArmed  = false
var bool prepShortArmed = false

var int lastPrepBar = na
prepOk = na(lastPrepBar) ? true : (bar_index - lastPrepBar) >= prepCooldown

stateLongOk  = not prepUseStateFilter or (state == "WAIT(L)")
stateShortOk = not prepUseStateFilter or (state == "WAIT(S)")

if usePrepBox and stateLongOk and (cciVal <= -prepExtreme)
    prepLongArmed := true
if usePrepBox and stateShortOk and (cciVal >= prepExtreme)
    prepShortArmed := true

longPrep  = usePrepBox and prepOk and stateLongOk  and prepLongArmed  and
ta.crossover(cciVal, -prepTrigger)
shortPrep = usePrepBox and prepOk and stateShortOk and prepShortArmed and
ta.crossunder(cciVal, prepTrigger)

if longPrep
    prepLongArmed := false
    lastPrepBar   := bar_index
if shortPrep
    prepShortArmed := false
```

```
    lastPrepBar    := bar_index

if prepLongArmed
    prepShortArmed := false
if prepShortArmed
    prepLongArmed := false
//
-------------------------------------------------------------------------------------------------------------------
// ENTRY GATE (CHOCH/BOS) — BREAK + RETEST OK olmadan ENTRY v1/v2 çalışmaz
//
-------------------------------------------------------------------------------------------------------------------
grpGate = "ENTRY Gate (CHOCH/BOS)"
useEntryGate     = input.bool(true,  "Filter ENTRY v1/v2 by CHOCH/BOS (gate)", group=grpGate)
gateNeedRetestOK = input.bool(true,  "Require RETEST OK before ENTRY",
group=grpGate)
gateMaxBars      = input.int(40,     "Gate max bars after BREAK", minval=5, maxval=500,
group=grpGate)

// Gate state (durum)
var int  gateDir       = 0     // 1=long, -1=short
var bool gateRetestOK  = false
var int  gateBreakBar  = na
var bool gateActive    = false
f_gatePass(_dir) =>
    not useEntryGate or (gateDir == 0 ? true : ((not gateNeedRetestOK or gateRetestOK) and
gateDir == _dir))

// Gate timeout (zaman aşımı) — gateMaxBars dolunca otomatik sıfırla
if useEntryGate and not na(gateBreakBar) and (bar_index - gateBreakBar) > gateMaxBars
    gateActive   := false
    gateRetestOK := false
    gateDir      := 0
    gateBreakBar := na
//
-------------------------------------------------------------------------------------------------------------------
// ENTRY Line (Gövde) + Retest  + 2-Candle Confirm (2. mum teyidi)
//
-------------------------------------------------------------------------------------------------------------------
grpEntry = "ENTRY Line (Gövde) + Retest"
entryEnable        = input.bool(true, "Enable ENTRY Retest", group=grpEntry)
entryExtendBars    = input.int(120, "Line Extend (bars)", minval=20, maxval=2000,
group=grpEntry)
entryNeedBearBull  = input.bool(true, "Require Candle Direction", group=grpEntry)
entryNeedPrep      = input.bool(false, "Require PREP on ENTRY bar", group=grpEntry)
entryUse2C         = input.bool(true, "Use 2-Candle Confirm (setup+confirm)", group=grpEntry)

entryATR = ta.atr(14)
tol      = entryATR * entryTol_eff
retrReq  = entryATR * entryRetr_eff

var line  entryLn       = na
var float entryPrice    = na
var int   entryDir      = 0
var bool  entryRetraced = false
var bool  entryFired    = false
var int   entryBar0     = na
```

```
var bool  setupArmed = false
var float setupHi    = na
var float setupLo    = na
var int   setupBar   = na

if entryEnable and flip and allowFlip and (bqMode != "HARD" or bqOk)
    if not na(entryLn)
        line.delete(entryLn)

    entryDir      := regTrend
    entryRetraced := false
    entryFired    := false
    entryBar0     := bar_index

    setupArmed := false
    setupHi    := na
    setupLo    := na
    setupBar   := na

    entryPrice := regTrend == -1 ? math.min(open, close) : math.max(open, close)

    entryLn := line.new(bar_index, entryPrice, bar_index + entryExtendBars, entryPrice,
      xloc=xloc.bar_index, extend=extend.none, color=color.new(#ff9900, 0), width=2,
style=line.style_solid)

if entryEnable and not na(entryLn)
    line.set_x2(entryLn, bar_index + entryExtendBars)

if entryEnable and entryDir == -1 and not entryRetraced and not na(entryPrice) and (bar_index >
entryBar0)
    if high >= (entryPrice + retrReq)
        entryRetraced := true

if entryEnable and entryDir == 1 and not entryRetraced and not na(entryPrice) and (bar_index >
entryBar0)
    if low <= (entryPrice - retrReq)
        entryRetraced := true

touchShort = entryEnable and entryDir == -1 and entryRetraced and not entryFired and not
na(entryPrice) and (high >= (entryPrice - tol))
rejShort   = touchShort and (close < entryPrice) and (not entryNeedBearBull or close < open)

touchLong  = entryEnable and entryDir == 1 and entryRetraced and not entryFired and not
na(entryPrice) and (low <= (entryPrice + tol))
rejLong    = touchLong and (close > entryPrice) and (not entryNeedBearBull or close > open)

// --- 2C Confirm (setup + confirm) ---
bool confirmShort = false
bool confirmLong  = false

if entryUse2C
    if barstate.isconfirmed and not setupArmed and rejShort and (not entryNeedPrep or shortPrep)
        setupArmed := true
        setupHi    := high
        setupLo    := low
        setupBar   := bar_index

    if barstate.isconfirmed and not setupArmed and rejLong and (not entryNeedPrep or longPrep)
        setupArmed := true
```

```
        setupHi    := high
        setupLo    := low
        setupBar   := bar_index

    confirmShort := barstate.isconfirmed and setupArmed and entryDir == -1 and (bar_index ==
setupBar + 1) and not na(setupLo) and (close < setupLo) and (not entryNeedPrep or shortPrep)
and f_gatePass(entryDir)
    confirmLong  := barstate.isconfirmed and setupArmed and entryDir == 1  and (bar_index ==
setupBar + 1) and not na(setupHi) and (close > setupHi) and (not entryNeedPrep or longPrep)  and
f_gatePass(entryDir)
    if confirmShort
        label.new(bar_index, na, "ENTRY", xloc=xloc.bar_index, yloc=yloc.abovebar,
          style=label.style_label_down, color=color.new(color.orange, 0), textcolor=color.black,
size=size.tiny)
        entryFired  := true
        setupArmed  := false
        if useEntryGate
            gateRetestOK := false
            gateActive   := false
            gateDir      := 0
            gateBreakBar := na

    if confirmLong
        label.new(bar_index, na, "ENTRY", xloc=xloc.bar_index, yloc=yloc.belowbar,
          style=label.style_label_up, color=color.new(color.orange, 0), textcolor=color.black,
size=size.tiny)
        entryFired  := true
        setupArmed  := false
        if useEntryGate
            gateRetestOK := false
            gateActive   := false
            gateDir      := 0
            gateBreakBar := na

    if setupArmed and not na(setupBar) and bar_index > setupBar + 1
        setupArmed := false

if not entryUse2C
    rejShortFire = barstate.isconfirmed and rejShort and (not entryNeedPrep or shortPrep) and
f_gatePass(entryDir)
    rejLongFire  = barstate.isconfirmed and rejLong  and (not entryNeedPrep or longPrep)  and
f_gatePass(entryDir)
    if rejShortFire
        label.new(bar_index, na, "ENTRY", xloc=xloc.bar_index, yloc=yloc.abovebar,
          style=label.style_label_down, color=color.new(color.orange, 0), textcolor=color.black,
size=size.tiny)
        entryFired := true

    if rejLongFire
        label.new(bar_index, na, "ENTRY", xloc=xloc.bar_index, yloc=yloc.belowbar,
          style=label.style_label_up, color=color.new(color.orange, 0), textcolor=color.black,
size=size.tiny)
        entryFired := true
// Flip anında kırılım kalitesini kaydet (ENTRY v1/v2 için ortak)
if flip and allowFlip and barstate.isconfirmed
    lastFlipBqOk  := bqOk
    lastFlipBqWhy := bqWhy
regSignalTxt = (flip and allowFlip) ? (regTrend == 1 ? TXT_BUY : TXT_SELL) : "—"
regTrendTxt  = regTrend == 1 ? TXT_UP : TXT_DOWN
```

```
if flip and allowFlip and barstate.isconfirmed
    if regTrend == 1
        label.new(bar_index, na, TXT_BUY,  xloc=xloc.bar_index, yloc=yloc.belowbar,
style=label.style_label_up,   color=color.new(color.lime, 0), textcolor=color.white, size=size.tiny)
    else
        label.new(bar_index, na, TXT_SELL, xloc=xloc.bar_index, yloc=yloc.abovebar,
style=label.style_label_down, color=color.new(color.red, 0),  textcolor=color.white, size=size.tiny)

//
——————————————————————————————————————————————————————————————————————————————
// ENTRY LINE (Body level) + Retest + Break = ENTRY  (v2) — AS IS
//
——————————————————————————————————————————————————————————————————————————————
grpEntry2 = "ENTRY Line (Body) + Retest Break (v2)"

entryEnable2    = input.bool(true,  "Enable ENTRY line + ENTRY label", group=grpEntry2)
entryMaxWait2   = input.int(30,     "Max bars to wait (after flip)",    minval=3, maxval=300,
group=grpEntry2)
entryLineBars2  = input.int(120,    "Line extend (bars)",               minval=20, maxval=2000,
group=grpEntry2)
entryNeedClose2 = input.bool(true,  "Confirm ENTRY on bar close",      group=grpEntry2)

var line  eLine2     = na
var float eLevel2    = na
var int   eDir2      = 0
var bool  eRetest2   = false
var int   eFlipBar2  = na
var bool  eActive2   = false

if entryEnable2 and flip and allowFlip and barstate.isconfirmed and (bqMode != "HARD" or bqOk)
    if not na(eLine2)
        line.delete(eLine2)

    eLevel2   := close
    eDir2     := regTrend
    eRetest2  := false
    eFlipBar2 := bar_index
    eActive2  := true

    eLine2 := line.new(x1=bar_index, y1=eLevel2, x2=bar_index + entryLineBars2, y2=eLevel2,
        xloc=xloc.bar_index, extend=extend.none, color=color.new(color.orange, 0), width=1,
style=line.style_solid)

if eActive2 and not na(eFlipBar2) and (bar_index - eFlipBar2) > entryMaxWait2
    eActive2 := false
    eDir2    := 0
    eRetest2 := false

if eActive2 and not na(eLine2) and not na(eLevel2)
    line.set_x2(eLine2, bar_index + entryLineBars2)
    line.set_y1(eLine2, eLevel2)
    line.set_y2(eLine2, eLevel2)

if eActive2 and eDir2 == -1 and not eRetest2 and not na(eLevel2)
    if high >= eLevel2
        eRetest2 := true

if eActive2 and eDir2 == 1 and not eRetest2 and not na(eLevel2)
```

```
    if low <= eLevel2
        eRetest2 := true

sellBreak2 = eActive2 and eDir2 == -1 and eRetest2 and not na(eLevel2) and ta.crossunder(close,
eLevel2)
buyBreak2  = eActive2 and eDir2 == 1  and eRetest2 and not na(eLevel2) and ta.crossover(close,
eLevel2)

conf2 = entryNeedClose2 ? barstate.isconfirmed : true

entrySell2 = conf2 and sellBreak2 and f_gatePass(eDir2)
entryBuy2  = conf2 and buyBreak2  and f_gatePass(eDir2)

if entrySell2
    label.new(bar_index, na, "ENTRY", xloc=xloc.bar_index, yloc=yloc.abovebar,
      style=label.style_label_down, color=color.new(color.orange, 0), textcolor=color.black,
size=size.tiny)
    eActive2 := false
    if useEntryGate
        gateRetestOK := false
        gateActive   := false
        gateDir      := 0
        gateBreakBar := na

if entryBuy2
    label.new(bar_index, na, "ENTRY", xloc=xloc.bar_index, yloc=yloc.belowbar,
      style=label.style_label_up, color=color.new(color.orange, 0), textcolor=color.black,
size=size.tiny)
    eActive2 := false
    if useEntryGate
        gateRetestOK := false
        gateActive   := false
        gateDir      := 0
        gateBreakBar := na
//
-------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------
// CHOCH + BOS (Auto) — Neckline (boyun çizgisi) + Retest + 2nd Cross = ENTRY
// NOT: Artık “BREAK” anında da balon basar (ENTRY beklemeye gerek yok).
//
-------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------
grpChoch = "CHOCH / BOS (Auto Neckline + Retest)"

chochEnable         = input.bool(true,  "Enable CHOCH/BOS Engine", group=grpChoch)
chochPivotLeft      = input.int(5,      "Pivot Left (swing)", minval=1, group=grpChoch)
chochPivotRight     = input.int(5,      "Pivot Right (swing)", minval=1, group=grpChoch)
chochMaxWaitBars    = input.int(40,     "Max wait bars (after break)", minval=5, maxval=500,
group=grpChoch)
chochLineBars       = input.int(200,    "Neckline extend (bars)", minval=20, maxval=2000,
group=grpChoch)
chochNeedClose      = input.bool(true,  "Confirm breaks on bar close", group=grpChoch)

// Disiplin filtreleri
chochOnlyTrend      = input.bool(true,  "Only TREND (REGIME!=RANGE)", group=grpChoch)
chochNeedAdxMin     = input.bool(true,  "Require ADX >= adxMin", group=grpChoch)
chochUseStateFilter = input.bool(true,  "Require STATE (WAIT(L)/WAIT(S))", group=grpChoch)

// YENİ: Balon kontrolleri
chochShowBreakLabel = input.bool(true,  "Show BREAK label (BOS/CHOCH)", group=grpChoch)
```

```
chochShowEntryLabel = input.bool(true,  "Show ENTRY label (after retest)", group=grpChoch)
chochDebugLabel     = input.bool(false, "DEBUG: show 1 label (why blocked)", group=grpChoch)

// Motorun çalışmasına izin (allow – izin)
chochOkRegime = not chochOnlyTrend or not isRange
chochOkAdx    = not chochNeedAdxMin or (adxVal >= adxMin)
chochAllowed  = chochEnable and chochOkRegime and chochOkAdx

// Swing (salınım) noktaları
cho_pH = ta.pivothigh(high, chochPivotLeft, chochPivotRight)
cho_pL = ta.pivotlow(low,  chochPivotLeft, chochPivotRight)

var float lastSwingH = na
var float lastSwingL = na
if not na(cho_pH)
    lastSwingH := cho_pH
if not na(cho_pL)
    lastSwingL := cho_pL

// Yapı yönü (structure direction – yapı yönü)
//  1: bullish (yükseliş yapısı), -1: bearish (düşüş yapısı)
var int structDir = 0
if structDir == 0
    if not na(lastSwingH) and close > lastSwingH
        structDir := 1
    if not na(lastSwingL) and close < lastSwingL
        structDir := -1

// Kırılım onayı (break confirmation – kırılım onayı)
bool brkOk = chochNeedClose ? barstate.isconfirmed : true

// STATE filtresi (disiplin kilidi)
choStateLongOk  = (not chochUseStateFilter) or (state == "WAIT(L)")
choStateShortOk = (not chochUseStateFilter) or (state == "WAIT(S)")

// BOS/CHOCH tespiti (yalnızca chochAllowed iken)
bosUp     = chochAllowed and brkOk and (structDir == 1)  and choStateLongOk  and not
na(lastSwingH) and ta.crossover(close, lastSwingH)   // crossover (yukarı kesişim)
bosDown   = chochAllowed and brkOk and (structDir == -1) and choStateShortOk and not
na(lastSwingL) and ta.crossunder(close, lastSwingL)  // crossunder (aşağı kesişim)

chochDown = chochAllowed and brkOk and (structDir == 1)  and choStateShortOk and not
na(lastSwingL) and ta.crossunder(close, lastSwingL)
chochUp   = chochAllowed and brkOk and (structDir == -1) and choStateLongOk  and not
na(lastSwingH) and ta.crossover(close, lastSwingH)

// Neckline (boyun çizgisi) motoru
var line   choLine     = na
var float  choLevel    = na
var int    choDir      = 0   // 1: long, -1: short
var bool   choRetested = false
var bool   choFired    = false
var int    choBreakBar = na
var bool   choActive   = false
var string choType     = ""  // "BOS" or "CHOCH"

// Yeni kırılım olunca çizgiyi kur
newBreak = (bosUp or bosDown or chochUp or chochDown)

if chochAllowed and newBreak and (bqMode != "HARD" or bqOk)
```

```
    if not na(choLine)
        line.delete(choLine)

    // yön ve tip
    if bosUp
        choDir    := 1
        choType   := "BOS"
        structDir := 1
    else if bosDown
        choDir    := -1
        choType   := "BOS"
        structDir := -1
    else if chochUp
        choDir    := 1
        choType   := "CHOCH"
        structDir := 1
    else if chochDown
        choDir    := -1
        choType   := "CHOCH"
        structDir := -1
    // ——— GATE: BREAK yakalandı → kapıyı aç, retest bekle (override engelli)
    if useEntryGate and not gateActive and not gateRetestOK
        gateDir      := choDir
        gateRetestOK := false
        gateBreakBar := bar_index
        gateActive   := true
    // kırılım mumunun gövde seviyesi (body level – gövde seviyesi)
    choLevel := choDir == 1 ? math.max(open, close) : math.min(open, close)

    choRetested := false
    choFired    := false
    choBreakBar := bar_index
    choActive   := true

    choLine := line.new(
         x1=bar_index, y1=choLevel,
         x2=bar_index + chochLineBars, y2=choLevel,
         xloc=xloc.bar_index, extend=extend.none,
         color=choDir == 1 ? color.new(color.aqua, 0) : color.new(color.fuchsia, 0),
         width=2, style=line.style_solid)

    // YENİ: BREAK balonu (ENTRY beklemeden)
    if chochShowBreakLabel
        string weakTag = (bqMode == "SOFT" and not bqOk) ? " • WEAK" : ""
        string brTxt = choDir == 1 ? ("▲ " + choType + " BREAK" + weakTag) : ("▼ " + choType + "
BREAK" + weakTag)
        label.new(bar_index, na, brTxt,
          xloc=xloc.bar_index,
          yloc=choDir == 1 ? yloc.belowbar : yloc.abovebar,
          style=choDir == 1 ? label.style_label_up : label.style_label_down,
          color=choDir == 1 ? color.new(color.aqua, 0) : color.new(color.fuchsia, 0),
          textcolor=color.black,
          size=size.tiny)

// çizgiyi uzat
if choActive and not na(choLine)
    line.set_x2(choLine, bar_index + chochLineBars)
    line.set_y1(choLine, choLevel)
    line.set_y2(choLine, choLevel)
```

```
// zaman aşımı
if choActive and not na(choBreakBar) and (bar_index - choBreakBar) > chochMaxWaitBars
    choActive   := false
    choRetested := false
    choFired    := false
    choDir      := 0
    choType     := ""

/// Retest şartı: fiyat çizgiye geri gelsin
// NOT: Fitil (wick/iğne) değil, GÖVDE (body/gövde) temasını retest say.
// LONG retest  : min(open,close) <= level  (gövde aşağı değdi)
// SHORT retest : max(open,close) >= level  (gövde yukarı değdi)
if choActive and not choRetested and not na(choLevel)
    if choDir == 1
        if math.min(open, close) <= choLevel
            choRetested := true

            // GATE: CHOCH retest OK → ENTRY serbest
            if useEntryGate and gateActive and gateDir == 1
                gateRetestOK := true
                gateActive   := false

    else if choDir == -1
        if math.max(open, close) >= choLevel
            choRetested := true

            // GATE: CHOCH retest OK → ENTRY serbest
            if useEntryGate and gateActive and gateDir == -1
                gateRetestOK := true
                gateActive   := false
// 2. kesişimde ENTRY
choEntryLong  = choActive and choRetested and not choFired and choDir == 1  and
ta.crossover(close, choLevel)
choEntryShort = choActive and choRetested and not choFired and choDir == -1 and
ta.crossunder(close, choLevel)

if choEntryLong and chochShowEntryLabel
    label.new(bar_index, na, "ENTRY " + choType,
      xloc=xloc.bar_index, yloc=yloc.belowbar,
      style=label.style_label_up, color=color.new(color.aqua, 0), textcolor=color.black, size=size.tiny)
    choFired  := true
    choActive := false

if choEntryShort and chochShowEntryLabel
    label.new(bar_index, na, "ENTRY " + choType,
      xloc=xloc.bar_index, yloc=yloc.abovebar,
      style=label.style_label_down, color=color.new(color.fuchsia, 0), textcolor=color.white,
size=size.tiny)
    choFired  := true
    choActive := false

// DEBUG (hata ayıklama): neden çalışmıyor? tek etikette göster
var label dbg = na
if chochDebugLabel and barstate.islast
    if not na(dbg)
        label.delete(dbg)

    string why = "CHOCH DBG | "
    why += "Allowed=" + str.tostring(chochAllowed)
    why += " | RegimeOK=" + str.tostring(chochOkRegime)
```

```
    why += " | AdxOK=" + str.tostring(chochOkAdx)
    why += " | State=" + state
    why += " | isRange=" + str.tostring(isRange)
    why += " | ADX=" + str.tostring(adxVal, "#.0")

    dbg := label.new(bar_index, na, why,
      xloc=xloc.bar_index, yloc=yloc.price,
      style=label.style_label_left, color=color.new(color.gray, 80), textcolor=color.white,
size=size.tiny)
 //
-----------------------------------------------------------------------------------------------------------------
// Liquidity Sweep + Retest + Breakout Confirmation (Likidite Süpürme + Geri Test + Kırılım Teyidi)
// NOT: Bu modül mevcut ST/ENTRY/CHOCH motorlarını değiştirmez. Sadece görsel uyarı basar.
//
-----------------------------------------------------------------------------------------------------------------
grpLiq = "Liquidity Sweep + Retest + Breakout (Ek Katman)"

// Aç/Kapa
liqEnable = input.bool(true, "Enable Liquidity Sweep module", group=grpLiq)

// Swing seviyesi (likidite havuzu) için pivotlar
liqPivotLeft  = input.int(5, "Sweep Pivot Left",  minval=1, group=grpLiq)
liqPivotRight = input.int(5, "Sweep Pivot Right", minval=1, group=grpLiq)

// Retest tolerans (tolerance – tolerans) ve zaman penceresi
liqTolAtrMult   = input.float(0.12, "Retest Tol (ATR x)", step=0.01, minval=0.0, maxval=2.0,
group=grpLiq)
liqMaxWaitBars  = input.int(25, "Max wait bars after SWEEP", minval=3, maxval=300,
group=grpLiq)

// Kibar etiketler
liqShowLabels = input.bool(true, "Show labels (small)", group=grpLiq)
liqShowBg     = input.bool(false, "Highlight background on BREAKOUT", group=grpLiq)

// Alarm (alert – uyarı)
liqAlerts = input.bool(false, "Enable alertconditions", group=grpLiq)

//
-----------------------------------------------------------------------------------------------------------------
// 1) Likidite seviyeleri: son swing high/low
//
-----------------------------------------------------------------------------------------------------------------
liq_pH = ta.pivothigh(high, liqPivotLeft, liqPivotRight)
liq_pL = ta.pivotlow(low,  liqPivotLeft, liqPivotRight)

var float liq_lastH = na
var float liq_lastL = na
if not na(liq_pH)
    liq_lastH := liq_pH
if not na(liq_pL)
    liq_lastL := liq_pL

// ATR tabanlı tolerans
liqATR = ta.atr(14)
liqTol = liqATR * liqTolAtrMult
```

```
//
// ---------------------------------------------------------------------------------------------------------
// 2) SWEEP tanımı:
//    - Bearish sweep (ayı süpürme): üst likidite alır -> high > lastH ve close < lastH
//    - Bullish sweep (boğa süpürme): alt likidite alır -> low < lastL ve close > lastL
//
// ---------------------------------------------------------------------------------------------------------
bearSweep = liqEnable and not na(liq_lastH) and (high > liq_lastH) and (close < liq_lastH)
bullSweep = liqEnable and not na(liq_lastL) and (low  < liq_lastL) and (close > liq_lastL)

//
// ---------------------------------------------------------------------------------------------------------
// 3) Durum makinesi (state machine – durum makinesi)
//    0: bekle
//    1: bull sweep oldu -> retest bekle -> breakout LONG teyidi
//   -1: bear sweep oldu -> retest bekle -> breakout SHORT teyidi
//
// ---------------------------------------------------------------------------------------------------------
var int   liqMode       = 0
var float liqLevel      = na
var int   liqSweepBar   = na
var bool  liqRetested   = false
var float liqRetHi      = na
var float liqRetLo      = na
var bool  liqBreakFired = false

// SWEEP yakalanınca kurulum
if liqEnable and (bullSweep or bearSweep)
    liqMode       := bullSweep ? 1 : -1
    liqLevel      := bullSweep ? liq_lastL : liq_lastH
    liqSweepBar   := bar_index
    liqRetested   := false
    liqRetHi      := na
    liqRetLo      := na
    liqBreakFired := false
    // GATE: SWEEP yakalandı → kapıyı aç, retest bekle (override engelli)
    if useEntryGate and not gateActive and not gateRetestOK
        gateDir      := bullSweep ? 1 : -1
        gateRetestOK := false
        gateBreakBar := bar_index
        gateActive   := true
    if liqShowLabels
        string sTxt = bullSweep ? (isTR ? "SWEEP (ALT)" : "SWEEP (LOW)") : (isTR ? "SWEEP (ÜST)"
: "SWEEP (HIGH)")
        label.new(bar_index, na, sTxt,
          xloc=xloc.bar_index,
          yloc=bullSweep ? yloc.belowbar : yloc.abovebar,
          style=bullSweep ? label.style_label_up : label.style_label_down,
          color=color.new(color.silver, 35),
          textcolor=color.black,
          size=size.tiny)

// Zaman aşımı
if liqEnable and liqMode != 0 and not na(liqSweepBar) and (bar_index - liqSweepBar) >
liqMaxWaitBars
    liqMode       := 0
```

```
    liqLevel     := na
    liqSweepBar  := na
    liqRetested  := false
    liqRetHi     := na
    liqRetLo     := na
    liqBreakFired := false

//
-------------------------------------------------------------------------------------------------------------------
// 4) Retest tanımı (gövde değil, seviyeye "yaklaşım" esas)
//    Bull: fiyat liqLevel çevresine iner (low <= level+tol) ve kapanış yukarıda kalır (close > level)
//    Bear: fiyat liqLevel çevresine çıkar (high >= level-tol) ve kapanış aşağıda kalır (close < level)
//
-------------------------------------------------------------------------------------------------------------------
bullRetest = liqEnable and liqMode == 1  and not liqRetested and not na(liqLevel) and (low  <=
(liqLevel + liqTol)) and (close > liqLevel)
bearRetest = liqEnable and liqMode == -1 and not liqRetested and not na(liqLevel) and (high >=
(liqLevel - liqTol)) and (close < liqLevel)

if bullRetest
    liqRetested := true

    // GATE: RETEST OK (LONG) → ENTRY serbest (gateActive false yapılır)
    if useEntryGate and gateActive and gateDir == 1
        gateRetestOK := true
        gateActive   := false

    liqRetHi := high
    liqRetLo := low

    if liqShowLabels
        label.new(bar_index, na, (isTR ? "RETEST OK" : "RETEST OK"),
          xloc=xloc.bar_index, yloc=yloc.belowbar,
          style=label.style_label_up,
          color=color.new(color.aqua, 55),
          textcolor=color.black,
          size=size.tiny)

if bearRetest
    liqRetested := true

    // GATE: RETEST OK (SHORT) → ENTRY serbest (gateActive false yapılır)
    if useEntryGate and gateActive and gateDir == -1
        gateRetestOK := true
        gateActive   := false

    liqRetHi := high
    liqRetLo := low

    if liqShowLabels
        label.new(bar_index, na, (isTR ? "RETEST OK" : "RETEST OK"),
          xloc=xloc.bar_index, yloc=yloc.abovebar,
          style=label.style_label_down,
          color=color.new(color.fuchsia, 55),
          textcolor=color.black,
          size=size.tiny)
```

```
//
// ---------------------------------------------------------------------------------------------------------
// 5) Breakout teyidi:
//    Bull: retest mumunun tepesinin üstünde kapanış -> close > liqRetHi
//    Bear: retest mumunun dibinin altında kapanış -> close < liqRetLo
//
// ---------------------------------------------------------------------------------------------------------
bullBreak = liqEnable and liqMode == 1 and liqRetested and not liqBreakFired and not
na(liqRetHi) and barstate.isconfirmed and (close > liqRetHi)
bearBreak = liqEnable and liqMode == -1 and liqRetested and not liqBreakFired and not
na(liqRetLo) and barstate.isconfirmed and (close < liqRetLo)

if bullBreak
    liqBreakFired := true
    liqMode := 0
    if liqShowLabels
        label.new(bar_index, na, (isTR ? "BREAKOUT LONG" : "BREAKOUT LONG"),
          xloc=xloc.bar_index, yloc=yloc.belowbar,
          style=label.style_label_up,
          color=color.new(color.lime, 35),
          textcolor=color.black,
          size=size.tiny)

    if useEntryGate
        gateRetestOK := false
        gateActive   := false
        gateDir      := 0
        gateBreakBar := na

if bearBreak
    liqBreakFired := true
    liqMode := 0
    if liqShowLabels
        label.new(bar_index, na, (isTR ? "BREAKOUT SHORT" : "BREAKOUT SHORT"),
          xloc=xloc.bar_index, yloc=yloc.abovebar,
          style=label.style_label_down,
          color=color.new(color.red, 35),
          textcolor=color.white,
          size=size.tiny)

    if useEntryGate
        gateRetestOK := false
        gateActive   := false
        gateDir      := 0
        gateBreakBar := na
// İsteğe bağlı arka plan vurgusu (çok kibar)
bgcol = liqShowBg ? (bullBreak ? color.new(color.lime, 92) : bearBreak ? color.new(color.red, 92) :
na) : na
bgcolor(bgcol)

// Alarm koşulları (TradingView alertcondition – alarm koşulu)
alertcondition(liqAlerts and bullSweep, title="SWEEP LOW", message="Bullish liquidity sweep
(ALT likidite süpürme) oluştu.")
alertcondition(liqAlerts and bearSweep, title="SWEEP HIGH", message="Bearish liquidity sweep
(ÜST likidite süpürme) oluştu.")
alertcondition(liqAlerts and bullRetest, title="RETEST OK (LONG)", message="Retest OK (geri test
tamam) – LONG yönü.")
```

```
alertcondition(liqAlerts and bearRetest, title="RETEST OK (SHORT)", message="Retest OK (geri test tamam) – SHORT yönü.")
alertcondition(liqAlerts and bullBreak, title="BREAKOUT LONG", message="Breakout confirmation (kırılım teyidi) – LONG.")
alertcondition(liqAlerts and bearBreak, title="BREAKOUT SHORT", message="Breakout confirmation (kırılım teyidi) – SHORT.")
//
// ═══════════════════════════════════════════════════════════════════════════
// NEW: ADX MODE (STATE filtreli + yön renkli + ok işaretli)
//
// ═══════════════════════════════════════════════════════════════════════════
string TXT_NO_TRADE_S = isTR ? "❌ SHORT ALINMAZ" : "❌ NO SHORT"
string TXT_WEAK_S     = isTR ? "⚠️ SHORT ZAYIF"  : "⚠️ WEAK SHORT"
string TXT_OK_S       = isTR ? "✅ SHORT ALINIR" : "✅ SHORT OK"
string TXT_CLEAN_S    = isTR ? "🟢 EN TEMİZ SHORT" : "🟢 CLEAN SHORT"
string TXT_LATE_S     = isTR ? "🟡 SHORT DEVAM – YENİ DİKKAT" : "🟡 SHORT ON – NEW CAUTION"

string TXT_NO_TRADE_L = isTR ? "❌ LONG ALINMAZ" : "❌ NO LONG"
string TXT_WEAK_L     = isTR ? "⚠️ LONG ZAYIF"   : "⚠️ WEAK LONG"
string TXT_OK_L       = isTR ? "✅ LONG ALINIR"  : "✅ LONG OK"
string TXT_CLEAN_L    = isTR ? "🟢 EN TEMİZ LONG" : "🟢 CLEAN LONG"
string TXT_LATE_L     = isTR ? "🟡 LONG DEVAM – YENİ DİKKAT" : "🟡 LONG ON – NEW CAUTION"

f_adxArrow(_adx) =>
    _adx > _adx[1] ? "↑" : _adx < _adx[1] ? "↓" : "→"

f_adxModeTxt(_adx, _dir, _state) =>
    // STATE filtresi: WAIT(S) değilse SHORT yazma, WAIT(L) değilse LONG yazma
    string out = "—"
    if _dir == -1
        if _state != "WAIT(S)"
            out := "—"
        else
            if _adx < adxT_no
                out := TXT_NO_TRADE_S
            else if _adx < adxT_weak
                out := TXT_WEAK_S
            else if _adx < adxT_ok
                out := TXT_OK_S
            else if _adx < adxT_clean
                out := TXT_CLEAN_S
            else
                out := TXT_LATE_S
    else if _dir == 1
        if _state != "WAIT(L)"
            out := "—"
        else
            if _adx < adxT_no
                out := TXT_NO_TRADE_L
            else if _adx < adxT_weak
                out := TXT_WEAK_L
```

```
        else if _adx < adxT_ok
            out := TXT_OK_L
        else if _adx < adxT_clean
            out := TXT_CLEAN_L
        else
            out := TXT_LATE_L
    out

f_adxModeBg(_adx, _dir, _state) =>
    // STATE uygun değilse gri
    color bg = color.new(color.gray, 85)

    bool stateBad = (_dir == -1 and _state != "WAIT(S)") or (_dir == 1 and _state != "WAIT(L)")
    if stateBad
        bg := color.new(color.gray, 85)
    else
        if _dir == -1
            if _adx < adxT_no
                bg := color.new(color.gray, 80)
            else if _adx < adxT_weak
                bg := color.new(color.red, 70)
            else if _adx < adxT_ok
                bg := color.new(color.maroon, 65)
            else if _adx < adxT_clean
                bg := color.new(color.red, 75)
            else
                bg := color.new(color.red, 82)
        else if _dir == 1
            if _adx < adxT_no
                bg := color.new(color.gray, 80)
            else if _adx < adxT_weak
                bg := color.new(color.green, 75)
            else if _adx < adxT_ok
                bg := color.new(color.teal, 75)
            else if _adx < adxT_clean
                bg := color.new(color.green, 80)
            else
                bg := color.new(color.lime, 82)
        else
            bg := color.new(color.gray, 85)

    bg


adxArrow   = f_adxArrow(adxVal)
adxModeTxt = f_adxModeTxt(adxVal, regTrend, state)
adxModeBg  = f_adxModeBg(adxVal, regTrend, state)

//
------------------------------------------------------------------------------------------------------------------------
// PANEL (NO AI ROWS) — 10 rows (0..9)
//
------------------------------------------------------------------------------------------------------------------------
var table t = table.new(position.top_right, 2, 11, border_width=0)

if barstate.islast
    color state_bg   = state == "WAIT(L)" ? color.new(#04f50c, 34) : state == "WAIT(S)" ?
color.new(#fd0909, 29) : color.new(#787b86, 68)
```

```
    color reg_bg     = regTrend == 1 ? color.new(upColor, 75) : color.new(downColor, 75)
    color regsig_bg  = (flip and allowFlip) ? color.new(color.yellow, 70) : color.new(color.gray, 85)
    color regime_bg  = isRange ? color.new(#ff9900, 43) : color.new(#00897b, 57)
    color exh_bg     = not useExhaustion ? color.new(color.gray, 85) : exhausting ?
color.new(#df40fb, 52) : color.new(color.gray, 85)

    table.cell(t, 0, 0, H_COIN, text_color=color.white, bgcolor=color.black, text_size=tblSize)
    table.cell(t, 1, 0, syminfo.basecurrency, text_color=color.white, bgcolor=color.black,
text_size=tblSize)

    table.cell(t, 0, 1, H_STATE, text_color=color.white, text_size=tblSize)
    table.cell(t, 1, 1, state, text_color=color.white, bgcolor=state_bg, text_size=tblSize)

    table.cell(t, 0, 2, H_REGSIG, text_color=color.white, text_size=tblSize)
    table.cell(t, 1, 2, regSignalTxt, text_color=color.white, bgcolor=regsig_bg, text_size=tblSize)

    table.cell(t, 0, 3, H_REGTREND, text_color=color.white, text_size=tblSize)
    table.cell(t, 1, 3, regTrendTxt, text_color=color.white, bgcolor=reg_bg, text_size=tblSize)

    table.cell(t, 0, 4, H_REGIME, text_color=color.white, text_size=tblSize)
    table.cell(t, 1, 4, regimeTxt, text_color=color.white, bgcolor=regime_bg, text_size=tblSize)

    table.cell(t, 0, 5, H_EXH, text_color=color.white, text_size=tblSize)
    table.cell(t, 1, 5, exhTxt, text_color=color.white, bgcolor=exh_bg, text_size=tblSize)

    table.cell(t, 0, 6, H_USDTD, text_color=color.white, text_size=tblSize)
    table.cell(t, 1, 6, usdt_text, text_color=color.white, bgcolor=usdt_bg, text_size=tblSize)

    table.cell(t, 0, 7, H_CCI, text_color=color.white, text_size=tblSize)
    table.cell(t, 1, 7, str.tostring(cciVal, "#.0"), text_color=color.white, text_size=tblSize)

    table.cell(t, 0, 8, H_RSI, text_color=color.white, text_size=tblSize)
    table.cell(t, 1, 8, str.tostring(rsiVal, "#.0"), text_color=color.white, text_size=tblSize)

    table.cell(t, 0, 9, H_ADX, text_color=color.white, text_size=tblSize)

    adxCellTxt = showAdxMode ? (str.tostring(adxVal, "#.0") + adxArrow + " | " + adxModeTxt) :
(str.tostring(adxVal, "#.0") + adxArrow)
    adxCellBg  = showAdxMode ? adxModeBg : color.new(color.gray, 85)

    table.cell(t, 1, 9, adxCellTxt, text_color=color.white, bgcolor=adxCellBg, text_size=tblSize)
      // MODE row (10. satır: index 10)
    if showModeRow
      table.cell(t, 0, 10, isTR ? "MOD" : "MODE", text_color=color.white, text_size=tblSize)
      table.cell(t, 1, 10, activeModeTxt, text_color=color.white,
        bgcolor=activeIsRange ? color.new(#ff9900, 49) : color.new(#00897b, 42), text_size=tblSize)
```